

# CrewAI Concepts: Crews

## Overview

A crew in crewAI represents a collaborative group of agents working together to achieve a set of tasks. Each crew defines the strategy for task execution, agent collaboration, and the overall workflow.

## Crew Attributes

Attribute - Parameters - Description

**Tasks** - tasks - A list of tasks assigned to the crew.

**Agents** - agents - A list of agents that are part of the crew.

**Process** `_(optional)_` - process - The process flow (e.g., sequential, hierarchical) the crew follows. Default is `sequential`.

**Verbose** `_(optional)_` - verbose - The verbosity level for logging during execution. Defaults to `False`.

**Manager LLM** `_(optional)_` - manager\_llm - The language model used by the manager agent in a hierarchical process. **Required when using a hierarchical process.**

**Function Calling LLM** `_(optional)_` - function\_calling\_llm - If passed, the crew will use this LLM to do function calling for tools for all agents in the crew. Each agent can have its own LLM, which overrides the crew's LLM for function calling.

**Config** `_(optional)_` - config - Optional configuration settings for the crew, in `Json` or `Dict[str, Any]` format.

**Max RPM** `_(optional)_` - max\_rpm - Maximum requests per minute the crew adheres to during execution. Defaults to `None`.

**Memory** `_(optional)_` - memory - Utilized for storing execution memories (short-term, long-term, entity memory). -

**Cache** `_(optional)_` - cache - Specifies whether to use a cache for storing the results of tools' execution. Defaults to `True`.

**Embedder** `_(optional)_` - embedder - Configuration for the embedder to be used by the crew. Mostly used by memory for now. Default is `{"provider": "openai"}`.

**Step Callback** `_(optional)_` - step\_callback - A function that is called after each step of every agent. This can be used to log the agent's actions or to perform other operations; it won't override the agent-specific `step_callback`.

**Task Callback** `_(optional)_` - task\_callback - A function that is called after the completion of each task. Useful for monitoring or additional operations post-task execution.

**Share Crew** `_(optional)_` - share\_crew - Whether you want to share the complete crew information and execution with the crewAI team to make the library better, and allow us to train models.

**Output Log File** `_(optional)_` - output\_log\_file - Set to `True` to save logs as `logs.txt` in the current directory or provide a file path. Logs will be in `JSON` format if the filename ends in `.json`, otherwise `.txt`. Defaults to `None`.

**Manager Agent** `_(optional)_` - manager\_agent - manager sets a custom agent that will be used as a manager.

**Prompt File** `_(optional)_` - prompt\_file - Path to the prompt `JSON` file to be used for the crew.

**Planning** (*optional*) - `planning` - Adds planning ability to the Crew. When activated before each Crew iteration, all Crew data is sent to an AgentPlanner that will plan the tasks and this plan will be added to each task description.

**Planning LLM** (*optional*) - `planning_llm` - The language model used by the AgentPlanner in a planning process.

**Knowledge Sources** (*optional*) - `knowledge_sources` - Knowledge sources available at the crew level, accessible to all the agents.

**Stream** (*optional*) - `stream` - Enable streaming output to receive real-time updates during crew execution. Returns a `CrewStreamingOutput` object that can be iterated for chunks. Defaults to `False`.

**Chat LLM** (*optional*) - `chat_llm` - The language model used to orchestrate `crewai chat CLI` interactions with the crew. Accepts a model name string or LLM instance. Defaults to `None`.

**Before Kickoff Callbacks** (*optional*) - `before_kickoff_callbacks` - A list of callable functions executed **before** the crew starts. Each callback receives and can modify the inputs dict. Distinct from the `@before_kickoff` decorator. Defaults to `[]`.

**After Kickoff Callbacks** (*optional*) - `after_kickoff_callbacks` - A list of callable functions executed **after** the crew finishes. Each callback receives and can modify the `CrewOutput`. Distinct from the `@after_kickoff` decorator. Defaults to `[]`.

**Tracing** (*optional*) - `tracing` - Controls tracing for the crew. `True` = always enable, `False` = always disable, `None` = inherit from environment / user settings. Defaults to `None`.

**Skills** (*optional*) - `skills` - A list of `Path` objects (skill search directories) or pre-loaded `Skill` objects applied to all agents in the crew. Defaults to `None`.

**Security Config** (*optional*) - `security_config` - A `SecurityConfig` instance managing crew fingerprinting and identity. Defaults to `SecurityConfig()`.

**Checkpoint** (*optional*) - `checkpoint` - Enables automatic checkpointing. Pass `True` for sensible defaults, a `CheckpointConfig` for full control, `False` to opt out, or `None` to inherit. See the Checkpointing section below. Defaults to `None`.

**Crew Max RPM:** The `max_rpm` attribute sets the maximum number of requests per minute the crew can perform to avoid rate limits and will override individual agents' `max_rpm` settings if you set it.

## Creating Crews

There are two common ways to create crews in CrewAI: using **JSONC project configuration (recommended for new crews)** or defining them **directly in code**.

### JSONC Configuration (Recommended)

New projects created with `crewai create crew` use `crew.jsonc` for crew-level settings and tasks, plus one file per agent in `agents/`.

`crewai run` automatically detects `crew.jsonc` or `crew.json`, loads the referenced agents, prompts for missing placeholders, and kicks off the crew.

### Example `crew.jsonc`

```
{
  "name": "Market Research Crew",
  "agents": ["researcher", "analyst"],
  "tasks": [
    {
      "name": "research",
      "description": "Research {topic} and collect the most relevant facts.",
      "expected_output": "Structured research notes about {topic}."
    }
  ]
}
```

```

    "agent": "researcher"
  },
  {
    "name": "analysis",
    "description": "Analyze the research and write a concise report.",
    "expected_output": "A markdown report with findings and recommendations.",
    "agent": "analyst",
    "context": ["research"],
    "output_file": "output/report.md"
  }
],
"process": "sequential",
"verbose": true,
"memory": true,
"inputs": {
  "topic": "AI Agents"
}
}

```

Each string in `agents` resolves to `agents/.jsonc` first, then `agents/.json`.

```

{
  "role": "{topic} Senior Researcher",
  "goal": "Find accurate and current information about {topic}.",
  "backstory": "You are a careful researcher who cites clear evidence.",
  "llm": "openai/gpt-4o",
  "tools": ["SerperDevTool"]
}

```

Tasks run in the order they appear in tasks when process is "sequential".

For hierarchical crews, set `"process": "hierarchical"` and provide either `manager_llm` or `manager_agent`. A `manager_agent` can reference an `agents/.jsonc` file that is not included in the top-level agents list.

JSON crew definitions support crew-level fields such as `process`, `verbose`, `memory`, `cache`, `max_rpm`, `planning`, `planning_llm`, `manager_llm`, `manager_agent`, `function_calling_llm`, `output_log_file`, `stream`, `tracing`, `before_kickoff_callbacks`, and `after_kickoff_callbacks`.

Python callbacks and custom classes use `{"python": "module.attribute"}`. Custom tools use `"custom:"` and load `tools/.py` at runtime.

Only run JSON crew projects from sources you trust. `custom: tools` and `{"python": "module.attribute"}` references execute local Python code when the crew loads.

## Classic Python/YAML Configuration

Classic projects created with `crewai create crew --classic` use `crew.py`, `config/agents.yaml`, `config/tasks.yaml`, and the `@CrewBase`, `@agent`, `@task`, and `@crew` decorators. That pattern remains supported and is documented in [Using Annotations](#).

## Direct Code Definition (Alternative)

Alternatively, you can define the crew directly in code without using YAML configuration files.

```

from crewai import Agent, Crew, Task, Process
from crewai_tools import YourCustomTool

class YourCrewName:
    def agent_one(self) -> Agent:
        return Agent(
            role="Data Analyst",
            goal="Analyze data trends in the market",
            backstory="An experienced data analyst with a background in economics",
            verbose=True,
            tools=[YourCustomTool()]
        )

    def agent_two(self) -> Agent:

```

```

    return Agent(
        role="Market Researcher",
        goal="Gather information on market dynamics",
        backstory="A diligent researcher with a keen eye for detail",
        verbose=True
    )

    def task_one(self) -> Task:
        return Task(
            description="Collect recent market data and identify trends.",
            expected_output="A report summarizing key trends in the market.",
            agent=self.agent_one()
        )

    def task_two(self) -> Task:
        return Task(
            description="Research factors affecting market dynamics.",
            expected_output="An analysis of factors influencing the market.",
            agent=self.agent_two()
        )

    def crew(self) -> Crew:
        return Crew(
            agents=[self.agent_one(), self.agent_two()],
            tasks=[self.task_one(), self.task_two()],
            process=Process.sequential,
            verbose=True
        )

```

How to run the above code:

```
YourCrewName().crew().kickoff(inputs={})
```

In this example:

- Agents and tasks are defined directly within the class without decorators.
- We manually create and manage the list of agents and tasks.
- This approach provides more control but can be less maintainable for larger projects.

## Crew Output

The output of a crew in the CrewAI framework is encapsulated within the `CrewOutput` class. This class provides a structured way to access results of the crew's execution, including various formats such as raw strings, JSON, and Pydantic models. The `CrewOutput` includes the results from the final task output, token usage, and individual task outputs.

### Crew Output Attributes

Attribute - Parameters - Type - Description

**Raw** - raw - str - The raw output of the crew. This is the default format for the output.

**Pydantic** - pydantic - Optional[BaseModel] - A Pydantic model object representing the structured output of the crew.

**JSON Dict** - json\_dict - Optional[Dict[str, Any]] - A dictionary representing the JSON output of the crew.

**Tasks Output** - tasks\_output - List[TaskOutput] - A list of TaskOutput objects, each representing the output of a task in the crew.

**Token Usage** - token\_usage - Dict[str, Any] - A summary of token usage, providing insights into the language model's performance during execution.

### Crew Output Methods and Properties

Method/Property - Description

**json** - Returns the JSON string representation of the crew output if the output format is JSON.

**to\_dict** - Converts the JSON and Pydantic outputs to a dictionary.

**\*\*str\*\*** - Returns the string representation of the crew output, prioritizing Pydantic, then JSON, then raw.

## Accessing Crew Outputs

Once a crew has been executed, its output can be accessed through the `output` attribute of the Crew object. The `CrewOutput` class provides various ways to interact with and present this output.

### Example

```
# Example crew execution
crew = Crew(
    agents=[research_agent, writer_agent],
    tasks=[research_task, write_article_task],
    verbose=True
)

crew_output = crew.kickoff()

# Accessing the crew output
print(f"Raw Output: {crew_output.raw}")
if crew_output.json_dict:
    print(f"JSON Output: {json.dumps(crew_output.json_dict, indent=2)}")
if crew_output.pydantic:
    print(f"Pydantic Output: {crew_output.pydantic}")
print(f"Tasks Output: {crew_output.tasks_output}")
print(f"Token Usage: {crew_output.token_usage}")
```

## Accessing Crew Logs

You can see real time log of the crew execution, by setting `output_log_file` as a `True(Boolean)` or a `file_name(str)`. Supports logging of events as both `file_name.txt` and `file_name.json`. In case of `True(Boolean)` will save as `logs.txt`.

In case of `output_log_file` is set as `False(Boolean)` or `None`, the logs will not be populated.

```
# Save crew logs
crew = Crew(output_log_file = True) # Logs will be saved as logs.txt
crew = Crew(output_log_file = file_name) # Logs will be saved as file_name.txt
crew = Crew(output_log_file = file_name.txt) # Logs will be saved as file_name.txt
crew = Crew(output_log_file = file_name.json) # Logs will be saved as file_name.json
```

## Checkpointing

Checkpointing lets a crew automatically save its state after key events (e.g. task completion) so that long-running or interrupted runs can be resumed exactly where they left off without re-executing completed tasks.

### Quick Start

Pass `checkpoint=True` to enable checkpointing with sensible defaults (saves to `.checkpoints/` after every task):

```
from crewai import Crew, Process

crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, write_task],
    process=Process.sequential,
    checkpoint=True, # saves to .checkpoints/ after every task
)

crew.kickoff(inputs={"topic": "AI trends"})
```

### Full Control with CheckpointConfig

Use `CheckpointConfig` for fine-grained control over location, trigger events, storage backend, and retention:

```
from crewai import Crew, Process
from crewai.state.checkpoint_config import CheckpointConfig

crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, write_task],
    process=Process.sequential,
    checkpoint=CheckpointConfig(
        location="./checkpoints",          # directory for JSON files (default)
        on_events=["task_completed"],      # trigger after each task (default)
        max_checkpoints=5,                 # keep only the 5 most recent checkpoints
    ),
)

crew.kickoff(inputs={"topic": "AI trends"})
```

## Resuming from a Checkpoint

Use `Crew.from_checkpoint()` to restore a crew from a saved checkpoint file, then call `kickoff()` to resume:

```
# Resume from the most recent checkpoint
crew = Crew.from_checkpoint("./checkpoints/latest.json")
crew.kickoff()
```

When restoring from a checkpoint, `checkpoint_inputs`, `checkpoint_train`, and `checkpoint_kickoff_event_id` are automatically reconstructed - you do not need to set these manually.

## CheckpointConfig *Attributes*

Attribute - Type - Default - Description

`location` - str - `"./checkpoints"` - Storage destination. For `JsonProvider` this is a directory path; for `SqliteProvider` a database file path.

`on_events` - list[str] - `["task_completed"]` - Event types that trigger a checkpoint write. Use `["*"]` to checkpoint on every event.

`provider` - `JsonProvider` \ - `SqliteProvider` - `JsonProvider()` - Storage backend. Defaults to `JsonProvider` (plain JSON files).

`max_checkpoints` - int \ - None - None - Maximum checkpoints to keep. Oldest are pruned after each write. None keeps all.

## Memory Utilization

Crews can utilize memory (short-term, long-term, and entity memory) to enhance their execution and learning over time. This feature allows crews to store and recall execution memories, aiding in decision-making and task execution strategies.

## Cache Utilization

Caches can be employed to store the results of tools' execution, making the process more efficient by reducing the need to re-execute identical tasks.

## Crew Usage Metrics

After the crew execution, you can access the `usage_metrics` attribute to view the language model (LLM) usage metrics for all tasks executed by the crew. This provides insights into operational efficiency and areas for improvement.

`total_tokens` is the billed total (`prompt_tokens` + `completion_tokens`). Breakdown fields such as `cached_prompt_tokens` and `cache_creation_tokens` describe subsets already included in those totals and are not added on top of `total_tokens`. See the **UsageMetrics field semantics** section in the Flows concept documentation for the full contract.

```
# Access the crew's usage metrics
crew = Crew(agents=[agent1, agent2], tasks=[task1, task2])
crew.kickoff()
print(crew.usage_metrics)
```

## Crew Execution Process

- **Sequential Process:** Tasks are executed one after another, allowing for a linear flow of work.
- **Hierarchical Process:** A manager agent coordinates the crew, delegating tasks and validating outcomes before proceeding. **Note:** A `manager_llm` or `manager_agent` is required for this process and it's essential for validating the process flow.

### *Kicking Off a Crew*

Once your crew is assembled, initiate the workflow with the `kickoff()` method. This starts the execution process according to the defined process flow.

```
# Start the crew's task execution
result = my_crew.kickoff()
print(result)
```

### *Different Ways to Kick Off a Crew*

Once your crew is assembled, initiate the workflow with the appropriate kickoff method. CrewAI provides several methods for better control over the kickoff process.

### *Synchronous Methods*

- `kickoff()`: Starts the execution process according to the defined process flow.
- `kickoff_for_each()`: Executes tasks sequentially for each provided input event or item in the collection.

### *Asynchronous Methods*

CrewAI offers two approaches for async execution:

Method - Type - Description

`akickoff()` - Native async - True async/await throughout the entire execution chain

`akickoff_for_each()` - Native async - Native async execution for each input in a list

`kickoff_async()` - Thread-based - Wraps synchronous execution in `asyncio.to_thread`

`kickoff_for_each_async()` - Thread-based - Thread-based async for each input in a list

For high-concurrency workloads, `akickoff()` and `akickoff_for_each()` are recommended as they use native async for task execution, memory operations, and knowledge retrieval.

```
# Start the crew's task execution
result = my_crew.kickoff()
print(result)

# Example of using kickoff_for_each
inputs_array = [{'topic': 'AI in healthcare'}, {'topic': 'AI in finance'}]
results = my_crew.kickoff_for_each(inputs=inputs_array)
for result in results:
    print(result)

# Example of using native async with akickoff
inputs = {'topic': 'AI in healthcare'}
async_result = await my_crew.akickoff(inputs=inputs)
```

```

print(async_result)

# Example of using native async with akickoff_for_each
inputs_array = [{'topic': 'AI in healthcare'}, {'topic': 'AI in finance'}]
async_results = await my_crew.ackickoff_for_each(inputs=inputs_array)
for async_result in async_results:
    print(async_result)

# Example of using thread-based kickoff_async
inputs = {'topic': 'AI in healthcare'}
async_result = await my_crew.kickoff_async(inputs=inputs)
print(async_result)

# Example of using thread-based kickoff_for_each_async
inputs_array = [{'topic': 'AI in healthcare'}, {'topic': 'AI in finance'}]
async_results = await my_crew.kickoff_for_each_async(inputs=inputs_array)
for async_result in async_results:
    print(async_result)

```

These methods provide flexibility in how you manage and execute tasks within your crew, allowing for both synchronous and asynchronous workflows tailored to your needs. For detailed async examples, see the [Kickoff Crew Asynchronously](#) guide.

## ***Streaming Crew Execution***

For real-time visibility into crew execution, you can enable streaming to receive output as it's generated:

```

# Enable streaming
crew = Crew(
    agents=[researcher],
    tasks=[task],
    stream=True
)

# Iterate over streaming output
streaming = crew.kickoff(inputs={"topic": "AI"})
for chunk in streaming:
    print(chunk.content, end="", flush=True)

# Access final result
result = streaming.result

```

Learn more about streaming in the [Streaming Crew Execution](#) guide.

## ***Replaying from a Specific Task***

You can now replay from a specific task using our CLI command `replay`.

The replay feature in CrewAI allows you to replay from a specific task using the command-line interface (CLI). By running the command `crewai replay -t`, you can specify the `task_id` for the replay process.

Kickoffs will now save the latest kickoffs returned task outputs locally for you to be able to replay from.

## ***Replaying from a Specific Task Using the CLI***

To use the replay feature, follow these steps:

1. Open your terminal or command prompt.
2. Navigate to the directory where your CrewAI project is located.
3. Run the following command:

To view the latest kickoff task IDs, use:

```
crewai log-tasks-outputs
```

Then, to replay from a specific task, use:

```
crewai replay -t <task_id>
```



These commands let you replay from your latest kickoff tasks, still retaining context from previously executed tasks.

# CrewAI Concepts: Tasks

## Overview

In the CrewAI framework, a Task is a specific assignment completed by an Agent.

Tasks provide all necessary details for execution, such as a description, the agent responsible, required tools, and more, facilitating a wide range of action complexities.

Tasks within CrewAI can be collaborative, requiring multiple agents to work together. This is managed through the task properties and orchestrated by the Crew's process, enhancing teamwork and efficiency.

CrewAI AMP includes a Visual Task Builder in Crew Studio that simplifies complex task creation and chaining. Design your task flows visually and test them in real-time without writing code.

!Task Builder Screenshot

The Visual Task Builder enables:

- Drag-and-drop task creation
- Visual task dependencies and flow
- Real-time testing and validation
- Easy sharing and collaboration

## Task Execution Flow

Tasks can be executed in two ways:

- **Sequential:** Tasks are executed in the order they are defined
- **Hierarchical:** Tasks are assigned to agents based on their roles and expertise

The execution flow is defined when creating the crew:

```
crew = Crew(  
    agents=[agent1, agent2],  
    tasks=[task1, task2],  
    process=Process.sequential # or Process.hierarchical  
)
```

## Task Attributes

Attribute - Parameters - Type - Description

**Description** - description - str - A clear, concise statement of what the task entails.

**Expected Output** - expected\_output - str - A detailed description of what the task's completion looks like.

**Name** \_(optional)\_ - name - Optional[str] - A name identifier for the task.

**Agent** \_(optional)\_ - agent - Optional[BaseAgent] - The agent responsible for executing the task.

**Tools** \_(optional)\_ - tools - List[BaseTool] - The tools/resources the agent is limited to use for this task.

**Context** \_(optional)\_ - context - Optional[List["Task"]] - Other tasks whose outputs will be used as context for this task.

**Async Execution** \_(optional)\_ - async\_execution - Optional[bool] - Whether the task should be executed asynchronously. Defaults to False.

**Human Input** \_(optional)\_ - human\_input - Optional[bool] - Whether the task should have a human review the final answer of the agent. Defaults to False.

**Markdown** \_(optional)\_ - markdown - Optional[bool] - Whether the task should instruct the agent to return the final answer formatted in Markdown. Defaults to False.

**Config** \_(optional)\_ - config - Optional[Dict[str, Any]] - Task-specific configuration parameters.

**Output File** \_(optional)\_ - output\_file - Optional[str] - File path for storing the task output.

**Create Directory** \_(optional)\_ - create\_directory - Optional[bool] - Whether to create the directory for output\_file if it doesn't exist. Defaults to True.

**Output JSON** \_(optional)\_ - output\_json - Optional[Type[BaseModel]] - A Pydantic model to structure the JSON output.

**Output Pydantic** \_(optional)\_ - output\_pydantic - Optional[Type[BaseModel]] - A Pydantic model for task output.

**Callback** \_(optional)\_ - callback - Optional[Any] - Function/object to be executed after task completion.

**Guardrail** \_(optional)\_ - guardrail - Optional[Callable] - Function to validate task output before proceeding to next task.

**Guardrails** \_(optional)\_ - guardrails - Optional[List[Callable]] - List of guardrails to validate task output before proceeding to next task.

**Guardrail Max Retries** \_(optional)\_ - guardrail\_max\_retries - Optional[int] - Maximum number of retries when guardrail validation fails. Defaults to 3.

The task attribute max\_retries is deprecated and will be removed in v1.0.0. Use guardrail\_max\_retries instead to control retry attempts when a guardrail fails.

## Creating Tasks

There are two common ways to create tasks in CrewAI: using **JSONC project configuration (recommended for new crews)** or defining them **directly in code**.

### ***JSONC Configuration (Recommended)***

New projects created with crewai create crew define tasks in crew.jsonc. The agents array points to files in agents/, and the tasks array defines the ordered work the crew should run.

After creating your CrewAI project as outlined in the Installation section, edit the generated crew.jsonc.

Use {placeholder} values in task description, expected\_output, and output\_file. Put defaults in the top-level inputs object; crewai run prompts for any missing values.

Here's an example crew.jsonc with two ordered tasks:

```
{
  "name": "Research Crew",
  "agents": ["researcher", "reporting_analyst"],
  "tasks": [
    {
      "name": "research_task",
      "description": "Conduct thorough research about {topic}. Include current and relevant information.",
      "expected_output": "A list of the most relevant information about {topic}.",
      "agent": "researcher"
    },
    {
      "name": "reporting_task",
      "description": "Review the research and expand it into a detailed report.",
      "expected_output": "A polished markdown report without fenced code blocks.",
      "agent": "reporting_analyst",
    }
  ]
}
```

```

        "context": ["research_task"],
        "markdown": true,
        "output_file": "report.md"
    }
},
"inputs": {
    "topic": "AI Agents"
}
}

```

Each task must include description and expected\_output. The agent value should match an agent name listed in agents. context is a list of prior task names; forward references are rejected so sequential context stays explicit.

Task entries support any public Task field. Common fields include name, agent, context, output\_file, tools, human\_input, async\_execution, guardrail, guardrails, guardrail\_max\_retries, markdown, input\_files, output\_json, output\_pydantic, response\_model, and converter\_cls. Use "type": "ConditionalTask" with a condition field for conditional tasks.

### ***Classic YAML Configuration***

Classic projects created with `crewai create crew --classic` use `config/tasks.yaml` and a `@CrewBase` class in `crew.py`. This remains supported for existing YAML projects or teams that prefer decorator-based Python wiring.

### ***Direct Code Definition (Alternative)***

Alternatively, you can define tasks directly in your code without using YAML configuration:

```

from crewai import Task

research_task = Task(
    description="""
        Conduct a thorough research about AI Agents.
        Make sure you find any interesting and relevant information given
        the current year is 2025.
    """,
    expected_output="""
        A list with 10 bullet points of the most relevant information about AI Agents
    """,
    agent=researcher
)

reporting_task = Task(
    description="""
        Review the context you got and expand each topic into a full section for a report.
        Make sure the report is detailed and contains any and all relevant information.
    """,
    expected_output="""
        A fully fledged reports with the main topics, each with a full section of information.
    """,
    agent=reporting_analyst,
    markdown=True, # Enable markdown formatting for the final output
    output_file="report.md"
)

```

Directly specify an agent for assignment or let the hierarchical CrewAI's process decide based on roles, availability, etc.

## **Task Output**

Understanding task outputs is crucial for building effective AI workflows. CrewAI provides a structured way to handle task results through the `TaskOutput` class, which supports multiple output formats and can be easily passed between tasks.

The output of a task in CrewAI framework is encapsulated within the TaskOutput class. This class provides a structured way to access results of a task, including various formats such as raw output, JSON, and Pydantic models.

By default, the TaskOutput will only include the raw output. A TaskOutput will only include the pydantic or json\_dict output if the original Task object was configured with output\_pydantic or output\_json, respectively.

### **Task Output Attributes**

Attribute - Parameters - Type - Description

**Description** - description - str - Description of the task.

**Summary** - summary - Optional[str] - Summary of the task, auto-generated from the first 10 words of the description.

**Raw** - raw - str - The raw output of the task. This is the default format for the output.

**Pydantic** - pydantic - Optional[BaseModel] - A Pydantic model object representing the structured output of the task.

**JSON Dict** - json\_dict - Optional[Dict[str, Any]] - A dictionary representing the JSON output of the task.

**Agent** - agent - str - The agent that executed the task.

**Output Format** - output\_format - OutputFormat - The format of the task output, with options including RAW, JSON, and Pydantic. The default is RAW.

**Messages** - messages - list[LLMMessage] - The messages from the last task execution.

### **Task Methods and Properties**

Method/Property - Description

**json** - Returns the JSON string representation of the task output if the output format is JSON.

**to\_dict** - Converts the JSON and Pydantic outputs to a dictionary.

**str** - Returns the string representation of the task output, prioritizing Pydantic, then JSON, then raw.

### **Accessing Task Outputs**

Once a task has been executed, its output can be accessed through the output attribute of the Task object. The TaskOutput class provides various ways to interact with and present this output.

### **Example**

```
# Example task
task = Task(
    description='Find and summarize the latest AI news',
    expected_output='A bullet list summary of the top 5 most important AI news',
    agent=research_agent,
    tools=[search_tool]
)

# Execute the crew
crew = Crew(
    agents=[research_agent],
    tasks=[task],
    verbose=True
)

result = crew.kickoff()

# Accessing the task output
task_output = task.output
```

```

print(f"Task Description: {task_output.description}")
print(f"Task Summary: {task_output.summary}")
print(f"Raw Output: {task_output.raw}")
if task_output.json_dict:
    print(f"JSON Output: {json.dumps(task_output.json_dict, indent=2)}")
if task_output.pydantic:
    print(f"Pydantic Output: {task_output.pydantic}")

```

## Markdown Output Formatting

The markdown parameter enables automatic markdown formatting for task outputs. When set to True, the task will instruct the agent to format the final answer using proper Markdown syntax.

### Using Markdown Formatting

```

# Example task with markdown formatting enabled
formatted_task = Task(
    description="Create a comprehensive report on AI trends",
    expected_output="A well-structured report with headers, sections, and bullet points",
    agent=reporter_agent,
    markdown=True # Enable automatic markdown formatting
)

```

When `markdown=True`, the agent will receive additional instructions to format the output using:

- `#` for headers
- **text** for bold text
- *text* for italic text
- `-` or `*` for bullet points
- ``code`` for inline code
- `language ```` for code blocks

### YAML Configuration with Markdown

```

analysis_task:
  description: >
    Analyze the market data and create a detailed report
  expected_output: >
    A comprehensive analysis with charts and key findings
  agent: analyst
  markdown: true # Enable markdown formatting
  output_file: analysis.md

```

### Benefits of Markdown Output

- **Consistent Formatting:** Ensures all outputs follow proper markdown conventions
- **Better Readability:** Structured content with headers, lists, and emphasis
- **Documentation Ready:** Output can be directly used in documentation systems
- **Cross-Platform Compatibility:** Markdown is universally supported

The markdown formatting instructions are automatically added to the task prompt when `markdown=True`, so you don't need to specify formatting requirements in your task description.

## Task Dependencies and Context

Tasks can depend on the output of other tasks using the `context` attribute. For example:

```

research_task = Task(
    description="Research the latest developments in AI",
    expected_output="A list of recent AI developments",
    agent=researcher
)

```

```
analysis_task = Task(
    description="Analyze the research findings and identify key trends",
    expected_output="Analysis report of AI trends",
    agent=analyst,
    context=[research_task] # This task will wait for research_task to complete
)
```

## Task Guardrails

Task guardrails provide a way to validate and transform task outputs before they are passed to the next task. This feature helps ensure data quality and provides feedback to agents when their output doesn't meet specific criteria.

CrewAI supports two types of guardrails:

1. **Function-based guardrails:** Python functions with custom validation logic, giving you complete control over the validation process and ensuring reliable, deterministic results.
2. **LLM-based guardrails:** String descriptions that use the agent's LLM to validate outputs based on natural language criteria. These are ideal for complex or subjective validation requirements.

### Function-Based Guardrails

To add a function-based guardrail to a task, provide a validation function through the `guardrail` parameter:

```
from typing import Tuple, Union, Dict, Any
from crewai import TaskOutput

def validate_blog_content(result: TaskOutput) -> Tuple[bool, Any]:
    """Validate blog content meets requirements."""
    try:
        # Check word count
        word_count = len(result.raw.split())
        if word_count > 200:
            return (False, "Blog content exceeds 200 words")

        # Additional validation logic here
        return (True, result.raw.strip())
    except Exception as e:
        return (False, "Unexpected error during validation")

blog_task = Task(
    description="Write a blog post about AI",
    expected_output="A blog post under 200 words",
    agent=blog_agent,
    guardrail=validate_blog_content # Add the guardrail function
)
```

### LLM-Based Guardrails (String Descriptions)

Instead of writing custom validation functions, you can use string descriptions that leverage LLM-based validation. When you provide a string to the `guardrail` or `guardrails` parameter, CrewAI automatically creates an `LLMGuardrail` that uses the agent's LLM to validate the output based on your description.

#### Requirements:

- The task must have an agent assigned (the guardrail uses the agent's LLM)
- Provide a clear, descriptive string explaining the validation criteria

```
from crewai import Task

# Single LLM-based guardrail
blog_task = Task(
    description="Write a blog post about AI",
    expected_output="A blog post under 200 words",
    agent=blog_agent,
    guardrail="The blog post must be under 200 words and contain no technical jargon"
)
```

LLM-based guardrails are particularly useful for:

- **Complex validation logic** that's difficult to express programmatically
- **Subjective criteria** like tone, style, or quality assessments
- **Natural language requirements** that are easier to describe than code

The LLM guardrail will:

1. Analyze the task output against your description
2. Return (True, output) if the output complies with the criteria
3. Return (False, feedback) with specific feedback if validation fails

#### Example with detailed validation criteria:

```
research_task = Task(
    description="Research the latest developments in quantum computing",
    expected_output="A comprehensive research report",
    agent=researcher_agent,
    guardrail="""
The research report must:
- Be at least 1000 words long
- Include at least 5 credible sources
- Cover both technical and practical applications
- Be written in a professional, academic tone
- Avoid speculation or unverified claims
    """
)
```

### Multiple Guardrails

You can apply multiple guardrails to a task using the `guardrails` parameter. Multiple guardrails are executed sequentially, with each guardrail receiving the output from the previous one. This allows you to chain validation and transformation steps.

The `guardrails` parameter accepts:

- A list of guardrail functions or string descriptions
- A single guardrail function or string (same as `guardrail`)

**Note:** If `guardrails` is provided, it takes precedence over `guardrail`. The `guardrail` parameter will be ignored when `guardrails` is set.

```
from typing import Tuple, Any
from crewai import TaskOutput, Task

def validate_word_count(result: TaskOutput) -> Tuple[bool, Any]:
    """Validate word count is within limits."""
    word_count = len(result.raw.split())
    if word_count < 100:
        return (False, f"Content too short: {word_count} words. Need at least 100 words.")
    if word_count > 500:
        return (False, f"Content too long: {word_count} words. Maximum is 500 words.")
    return (True, result.raw)

def validate_no_profanity(result: TaskOutput) -> Tuple[bool, Any]:
    """Check for inappropriate language."""
    profanity_words = ["badword1", "badword2"] # Example list
    content_lower = result.raw.lower()
    for word in profanity_words:
        if word in content_lower:
            return (False, f"Inappropriate language detected: {word}")
    return (True, result.raw)

def format_output(result: TaskOutput) -> Tuple[bool, Any]:
    """Format and clean the output."""
    formatted = result.raw.strip()
    # Capitalize first letter
```



```

        formatted = formatted[0].upper() + formatted[1:] if formatted else formatted
        return (True, formatted)

# Apply multiple guardrails sequentially
blog_task = Task(
    description="Write a blog post about AI",
    expected_output="A well-formatted blog post between 100-500 words",
    agent=blog_agent,
    guardrails=[
        validate_word_count,      # First: validate length
        validate_no_profanity,     # Second: check content
        format_output              # Third: format the result
    ],
    guardrail_max_retries=3
)

```

In this example, the guardrails execute in order:

1. `validate_word_count` checks the word count
2. `validate_no_profanity` checks for inappropriate language (using the output from step 1)
3. `format_output` formats the final result (using the output from step 2)

If any guardrail fails, the error is sent back to the agent, and the task is retried up to `guardrail_max_retries` times.

### Mixing function-based and LLM-based guardrails:

You can combine both function-based and string-based guardrails in the same list:

```

from typing import Tuple, Any
from crewai import TaskOutput, Task

def validate_word_count(result: TaskOutput) -> Tuple[bool, Any]:
    """Validate word count is within limits."""
    word_count = len(result.raw.split())
    if word_count < 100:
        return (False, f"Content too short: {word_count} words. Need at least 100 words.")
    if word_count > 500:
        return (False, f"Content too long: {word_count} words. Maximum is 500 words.")
    return (True, result.raw)

# Mix function-based and LLM-based guardrails
blog_task = Task(
    description="Write a blog post about AI",
    expected_output="A well-formatted blog post between 100-500 words",
    agent=blog_agent,
    guardrails=[
        validate_word_count, # Function-based: precise word count check
        "The content must be engaging and suitable for a general audience", # LLM-based: subjective quality c
        "The writing style should be clear, concise, and free of technical jargon" # LLM-based: style validat
    ],
    guardrail_max_retries=3
)

```

This approach combines the precision of programmatic validation with the flexibility of LLM-based assessment for subjective criteria.

### Guardrail Function Requirements

#### 1. Function Signature:

- Must accept exactly one parameter (the task output)
- Should return a tuple of (`bool`, `Any`)
- Type hints are recommended but optional

#### 2. Return Values:

- On success: it returns a tuple of (`bool`, `Any`). For example: (`True`, `validated_result`)

- On Failure: it returns a tuple of (bool, str). For example: (False, "Error message explain the failure")

## ***Error Handling Best Practices***

### **1. Structured Error Responses:**

```
from crewai import TaskOutput, LLMGuardrail

def validate_with_context(result: TaskOutput) -> Tuple[bool, Any]:
    try:
        # Main validation logic
        validated_data = perform_validation(result)
        return (True, validated_data)
    except ValidationError as e:
        return (False, f"VALIDATION_ERROR: {str(e)}")
    except Exception as e:
        return (False, str(e))
```

### **2. Error Categories:**

- Use specific error codes
- Include relevant context
- Provide actionable feedback

### **3. Validation Chain:**

```
from typing import Any, Dict, List, Tuple, Union
from crewai import TaskOutput

def complex_validation(result: TaskOutput) -> Tuple[bool, Any]:
    """Chain multiple validation steps."""
    # Step 1: Basic validation
    if not result:
        return (False, "Empty result")

    # Step 2: Content validation
    try:
        validated = validate_content(result)
        if not validated:
            return (False, "Invalid content")

        # Step 3: Format validation
        formatted = format_output(validated)
        return (True, formatted)
    except Exception as e:
        return (False, str(e))
```

## ***Handling Guardrail Results***

When a guardrail returns (False, error):

1. The error is sent back to the agent
2. The agent attempts to fix the issue
3. The process repeats until:
  - The guardrail returns (True, result)
  - Maximum retries are reached (guardrail\_max\_retries)

Example with retry handling:

```
from typing import Optional, Tuple, Union
from crewai import TaskOutput, Task

def validate_json_output(result: TaskOutput) -> Tuple[bool, Any]:
    """Validate and parse JSON output."""
    try:
        # Try to parse as JSON
```

```

        data = json.loads(result)
        return (True, data)
    except json.JSONDecodeError as e:
        return (False, "Invalid JSON format")

task = Task(
    description="Generate a JSON report",
    expected_output="A valid JSON object",
    agent=analyst,
    guardrail=validate_json_output,
    guardrail_max_retries=3 # Limit retry attempts
)

```

## Getting Structured Consistent Outputs from Tasks

It's also important to note that the output of the final task of a crew becomes the final output of the actual crew itself.

### *Using* output\_pydantic

The `output_pydantic` property allows you to define a Pydantic model that the task output should conform to. This ensures that the output is not only structured but also validated according to the Pydantic model.

Here's an example demonstrating how to use `output_pydantic`:

```

import json

from crewai import Agent, Crew, Process, Task
from pydantic import BaseModel

class Blog(BaseModel):
    title: str
    content: str

blog_agent = Agent(
    role="Blog Content Generator Agent",
    goal="Generate a blog title and content",
    backstory="""You are an expert content creator, skilled in crafting engaging and informative blog posts.""",
    verbose=False,
    allow_delegation=False,
    llm="gpt-4o",
)

task1 = Task(
    description="""Create a blog title and content on a given topic. Make sure the content is under 200 words.
    expected_output="A compelling blog title and well-written content.",
    agent=blog_agent,
    output_pydantic=Blog,
)

# Instantiate your crew with a sequential process
crew = Crew(
    agents=[blog_agent],
    tasks=[task1],
    verbose=True,
    process=Process.sequential,
)

result = crew.kickoff()

# Option 1: Accessing Properties Using Dictionary-Style Indexing
print("Accessing Properties - Option 1")
title = result["title"]
content = result["content"]
print("Title:", title)
print("Content:", content)

# Option 2: Accessing Properties Directly from the Pydantic Model
print("Accessing Properties - Option 2")
title = result.pydantic.title

```

```

content = result.pydantic.content
print("Title:", title)
print("Content:", content)

# Option 3: Accessing Properties Using the to_dict() Method
print("Accessing Properties - Option 3")
output_dict = result.to_dict()
title = output_dict["title"]
content = output_dict["content"]
print("Title:", title)
print("Content:", content)

# Option 4: Printing the Entire Blog Object
print("Accessing Properties - Option 5")
print("Blog:", result)

```

In this example:

- A Pydantic model Blog is defined with title and content fields.
- The task task1 uses the output\_pydantic property to specify that its output should conform to the Blog model.
- After executing the crew, you can access the structured output in multiple ways as shown.

### ***Explanation of Accessing the Output***

1. Dictionary-Style Indexing: You can directly access the fields using result["field\_name"]. This works because the CrewOutput class implements the **getitem** method.
2. Directly from Pydantic Model: Access the attributes directly from the result.pydantic object.
3. Using to\_dict() Method: Convert the output to a dictionary and access the fields.
4. Printing the Entire Object: Simply print the result object to see the structured output.

### ***Using output\_json***

The output\_json property allows you to define the expected output in JSON format. This ensures that the task's output is a valid JSON structure that can be easily parsed and used in your application.

Here's an example demonstrating how to use output\_json:

```

import json

from crewai import Agent, Crew, Process, Task
from pydantic import BaseModel

# Define the Pydantic model for the blog
class Blog(BaseModel):
    title: str
    content: str

# Define the agent
blog_agent = Agent(
    role="Blog Content Generator Agent",
    goal="Generate a blog title and content",
    backstory="""You are an expert content creator, skilled in crafting engaging and informative blog posts.""",
    verbose=False,
    allow_delegation=False,
    llm="gpt-4o",
)

# Define the task with output_json set to the Blog model
task1 = Task(
    description="""Create a blog title and content on a given topic. Make sure the content is under 200 words. expected_output="A JSON object with 'title' and 'content' fields.",
    agent=blog_agent,
    output_json=Blog,
)

```

```

# Instantiate the crew with a sequential process
crew = Crew(
    agents=[blog_agent],
    tasks=[task1],
    verbose=True,
    process=Process.sequential,
)

# Kickoff the crew to execute the task
result = crew.kickoff()

# Option 1: Accessing Properties Using Dictionary-Style Indexing
print("Accessing Properties - Option 1")
title = result["title"]
content = result["content"]
print("Title:", title)
print("Content:", content)

# Option 2: Printing the Entire Blog Object
print("Accessing Properties - Option 2")
print("Blog:", result)

```

In this example:

- A Pydantic model `Blog` is defined with `title` and `content` fields, which is used to specify the structure of the JSON output.
- The task `task1` uses the `output_json` property to indicate that it expects a JSON output conforming to the `Blog` model.
- After executing the crew, you can access the structured JSON output in two ways as shown.

### ***Explanation of Accessing the Output***

1. **Accessing Properties Using Dictionary-Style Indexing:** You can access the fields directly using `result["field_name"]`. This is possible because the `CrewOutput` class implements the **`getitem`** method, allowing you to treat the output like a dictionary. In this option, we're retrieving the `title` and `content` from the `result`.
2. **Printing the Entire Blog Object:** By printing `result`, you get the string representation of the `CrewOutput` object. Since the **`str`** method is implemented to return the JSON output, this will display the entire output as a formatted string representing the `Blog` object.

---

By using `output_pydantic` or `output_json`, you ensure that your tasks produce outputs in a consistent and structured format, making it easier to process and utilize the data within your application or across multiple tasks.

## **Integrating Tools with Tasks**

Leverage tools from the CrewAI Toolkit and LangChain Tools for enhanced task performance and agent interaction.

### **Creating a Task with Tools**

```

import os
os.environ["OPENAI_API_KEY"] = "Your Key"
os.environ["SERPER_API_KEY"] = "Your Key" # serper.dev API key

from crewai import Agent, Task, Crew
from crewai_tools import SerperDevTool

research_agent = Agent(
    role='Researcher',
    goal='Find and summarize the latest AI news',
    backstory="""You're a researcher at a large company.

```

```

    You're responsible for analyzing data and providing insights
    to the business.""",
    verbose=True
)

# to perform a semantic search for a specified query from a text's content across the internet
search_tool = SerperDevTool()

task = Task(
    description='Find and summarize the latest AI news',
    expected_output='A bullet list summary of the top 5 most important AI news',
    agent=research_agent,
    tools=[search_tool]
)

crew = Crew(
    agents=[research_agent],
    tasks=[task],
    verbose=True
)

result = crew.kickoff()
print(result)

```

This demonstrates how tasks with specific tools can override an agent's default set for tailored task execution.

## Referring to Other Tasks

In CrewAI, the output of one task is automatically relayed into the next one, but you can specifically define what tasks' output, including multiple, should be used as context for another task.

This is useful when you have a task that depends on the output of another task that is not performed immediately after it. This is done through the `context` attribute of the task:

```

# ...

research_ai_task = Task(
    description="Research the latest developments in AI",
    expected_output="A list of recent AI developments",
    async_execution=True,
    agent=research_agent,
    tools=[search_tool]
)

research_ops_task = Task(
    description="Research the latest developments in AI Ops",
    expected_output="A list of recent AI Ops developments",
    async_execution=True,
    agent=research_agent,
    tools=[search_tool]
)

write_blog_task = Task(
    description="Write a full blog post about the importance of AI and its latest news",
    expected_output="Full blog post that is 4 paragraphs long",
    agent=writer_agent,
    context=[research_ai_task, research_ops_task]
)

#...

```

## Asynchronous Execution

You can define a task to be executed asynchronously. This means that the crew will not wait for it to be completed to continue with the next task. This is useful for tasks that take a long time to be completed, or that are not crucial for the next tasks to be performed.

You can then use the `context` attribute to define in a future task that it should wait for the output of the asynchronous task to be completed.

```
#...

list_ideas = Task(
    description="List of 5 interesting ideas to explore for an article about AI.",
    expected_output="Bullet point list of 5 ideas for an article.",
    agent=researcher,
    async_execution=True # Will be executed asynchronously
)

list_important_history = Task(
    description="Research the history of AI and give me the 5 most important events.",
    expected_output="Bullet point list of 5 important events.",
    agent=researcher,
    async_execution=True # Will be executed asynchronously
)

write_article = Task(
    description="Write an article about AI, its history, and interesting ideas.",
    expected_output="A 4 paragraph article about AI.",
    agent=writer,
    context=[list_ideas, list_important_history] # Will wait for the output of the two tasks to be completed
)

#...
```

## Callback Mechanism

The callback function is executed after the task is completed, allowing for actions or notifications to be triggered based on the task's outcome.

```
# ...

def callback_function(output: TaskOutput):
    # Do something after the task is completed
    # Example: Send an email to the manager
    print(f"""
        Task completed!
        Task: {output.description}
        Output: {output.raw}
    """)

research_task = Task(
    description='Find and summarize the latest AI news',
    expected_output='A bullet list summary of the top 5 most important AI news',
    agent=research_agent,
    tools=[search_tool],
    callback=callback_function
)

#...
```

## Accessing a Specific Task Output

Once a crew finishes running, you can access the output of a specific task by using the `output` attribute of the task object:

```
# ...
task1 = Task(
    description='Find and summarize the latest AI news',
    expected_output='A bullet list summary of the top 5 most important AI news',
    agent=research_agent,
    tools=[search_tool]
)

#...

crew = Crew(
    agents=[research_agent],
    tasks=[task1, task2, task3],
    verbose=True
)

result = crew.kickoff()
```

```
# Returns a TaskOutput object with the description and results of the task
print(f"""
    Task completed!
    Task: {task1.output.description}
    Output: {task1.output.raw}
    """)
```

## Tool Override Mechanism

Specifying tools in a task allows for dynamic adaptation of agent capabilities, emphasizing CrewAI's flexibility.

## Error Handling and Validation Mechanisms

While creating and executing tasks, certain validation mechanisms are in place to ensure the robustness and reliability of task attributes. These include but are not limited to:

- Ensuring only one output type is set per task to maintain clear output expectations.
- Preventing the manual assignment of the `id` attribute to uphold the integrity of the unique identifier system.

These validations help in maintaining the consistency and reliability of task executions within the crewAI framework.

## Creating Directories when Saving Files

The `create_directory` parameter controls whether CrewAI should automatically create directories when saving task outputs to files. This feature is particularly useful for organizing outputs and ensuring that file paths are correctly structured, especially when working with complex project hierarchies.

### ***Default Behavior***

By default, `create_directory=True`, which means CrewAI will automatically create any missing directories in the output file path:

```
# Default behavior - directories are created automatically
report_task = Task(
    description='Generate a comprehensive market analysis report',
    expected_output='A detailed market analysis with charts and insights',
    agent=analyst_agent,
    output_file='reports/2025/market_analysis.md', # Creates 'reports/2025/' if it doesn't exist
    markdown=True
)
```

### ***Disabling Directory Creation***

If you want to prevent automatic directory creation and ensure that the directory already exists, set `create_directory=False`:

```
# Strict mode - directory must already exist
strict_output_task = Task(
    description='Save critical data that requires existing infrastructure',
    expected_output='Data saved to pre-configured location',
    agent=data_agent,
    output_file='secure/vault/critical_data.json',
    create_directory=False # Will raise RuntimeError if 'secure/vault/' doesn't exist
)
```

## YAML Configuration

You can also configure this behavior in your YAML task definitions:

```
analysis_task:
  description: >
    Generate quarterly financial analysis
  expected_output: >
    A comprehensive financial report with quarterly insights
```



```
agent: financial_analyst
output_file: reports/quarterly/q4_2024_analysis.pdf
create_directory: true # Automatically create 'reports/quarterly/' directory
```

```
audit_task:
  description: >
    Perform compliance audit and save to existing audit directory
  expected_output: >
    A compliance audit report
  agent: auditor
  output_file: audit/compliance_report.md
  create_directory: false # Directory must already exist
```

## Use Cases

### Automatic Directory Creation (create\_directory=True):

- Development and prototyping environments
- Dynamic report generation with date-based folders
- Automated workflows where directory structure may vary
- Multi-tenant applications with user-specific folders

### Manual Directory Management (create\_directory=False):

- Production environments with strict file system controls
- Security-sensitive applications where directories must be pre-configured
- Systems with specific permission requirements
- Compliance environments where directory creation is audited

## Error Handling

When create\_directory=False and the directory doesn't exist, CrewAI will raise a `RuntimeError`:

```
try:
    result = crew.kickoff()
except RuntimeError as e:
    # Handle missing directory error
    print(f"Directory creation failed: {e}")
    # Create directory manually or use fallback location
```

Check out the video below to see how to use structured outputs in CrewAI:

## Conclusion

Tasks are the driving force behind the actions of agents in CrewAI. By properly defining tasks and their outcomes, you set the stage for your AI agents to work effectively, either independently or as a collaborative unit. Equipping tasks with appropriate tools, understanding the execution process, and following robust validation practices are crucial for maximizing CrewAI's potential, ensuring agents are effectively prepared for their assignments and that tasks are executed as intended.

# CrewAI Concepts: Processes

## Overview

Processes orchestrate the execution of tasks by agents, akin to project management in human teams. These processes ensure tasks are distributed and executed efficiently, in alignment with a predefined strategy.

## Process Implementations

- **Sequential:** Executes tasks sequentially, ensuring tasks are completed in an orderly progression.
- **Hierarchical:** Organizes tasks in a managerial hierarchy, where tasks are delegated and executed based on a structured chain of command. A manager language model (`manager_llm`) or a custom manager agent (`manager_agent`) must be specified in the crew to enable the hierarchical process, facilitating the creation and management of tasks by the manager.

## The Role of Processes in Teamwork

Processes enable individual agents to operate as a cohesive unit, streamlining their efforts to achieve common objectives with efficiency and coherence.

## Assigning Processes to a Crew

To assign a process to a crew, specify the process type upon crew creation to set the execution strategy. For a hierarchical process, ensure to define `manager_llm` or `manager_agent` for the manager agent.

```
from crewai import Crew, Process

# Example: Creating a crew with a sequential process
crew = Crew(
    agents=my_agents,
    tasks=my_tasks,
    process=Process.sequential
)

# Example: Creating a crew with a hierarchical process
# Ensure to provide a manager_llm or manager_agent
crew = Crew(
    agents=my_agents,
    tasks=my_tasks,
    process=Process.hierarchical,
    manager_llm="gpt-4o"
    # or
    # manager_agent=my_manager_agent
)
```

**Note:** Ensure `my_agents` and `my_tasks` are defined prior to creating a `Crew` object, and for the hierarchical process, either `manager_llm` or `manager_agent` is also required.

## Sequential Process

This method mirrors dynamic team workflows, progressing through tasks in a thoughtful and systematic manner. Task execution follows the predefined order in the task list, with the output of one task serving as context for the next.

To customize task context, utilize the `context` parameter in the `Task` class to specify outputs that should be used as context for subsequent tasks.

## Hierarchical Process

Emulates a corporate hierarchy, CrewAI allows specifying a custom manager agent or automatically creates one, requiring the specification of a manager language model (`manager_llm`). This agent oversees task execution, including planning, delegation, and validation. Tasks are not pre-assigned; the manager allocates tasks to agents based on their capabilities, reviews outputs, and assesses task completion.

## **Process Class: Detailed Overview**

The Process class is implemented as an enumeration (Enum), ensuring type safety and restricting process values to the defined types (`sequential`, `hierarchical`).

## **Conclusion**

The structured collaboration facilitated by processes within CrewAI is crucial for enabling systematic teamwork among agents. This documentation has been updated to reflect the latest features and enhancements, ensuring users have access to the most current and comprehensive information.